

Utilizarea intreruperilor si transferul de date intre microcontrolere cu ajutorul interfetei seriale

1 Scopul lucrarii

Studentul ar trebui ca, la sfarsitul sedintei de laborator, sa aiba cunostintele de baza despre tehnica intreruperilor si utilizarea acestora in comunicarea dintre doua microcontrolere.

2 Introducere

In acest capitol vor fi prezentate:

2.1. Sursele de intreruperi puse la dispozitie de microcontroler.

2.1 Intreruperile

Nucleul CIP51 pune la dispozitia utilizatorului un sistem extins de intreruperi cu 20 de surse, cu doua nivele de prioritate. Fiecarei intreruperi ii este asociat un fanion aflat in registrele SFR. In momentul in care sunt indeplinite conditiile unei intreruperi valide, acest fanion este setat.

Daca intreruperile sunt activate pentru o anumita sursa, o cerere de intrerupere (IRQ) este generata cand fanionul de intrerupere este setat. Deodata ce executia instructiunii curente ia sfarsit, nucleul de procesare genereaza un apel (LCALL) catre o adresa din memoria de cod asociata intreruperii, pentru a executa procedura de deservire a intreruperii (ISR). Fiecare ISR trebuie sa se incheie cu o instructiune RETI, care duce programul la urmatoarea instructiune care ar fi fost executata daca nu ar fi avut loc intreruperea. Daca intreruperile nu sunt activate, setarea fanionului de intrerupere este ignorata, iar programul este executat in continuare. Fanionul de intrerupere este setat indiferent daca intreruperea este activata sau dezactivata.

Fiecare intrerupere poate fi activata sau dezactivata separat, cu ajutorul bitului de activare asociat, situat intr-un SFR (IE-EIE2). Cu toate acestea, cererile de intrerupere trebuie activate global prin setarea bitului IE.7 (EA-Interrupt Enable Bit), inainte ca intreruperile individuale sa fie recunoscute de nucleul de procesare. Resetarea bitului EA dezactiveaza toate sursele de intrerupere, indiferent de starea lor proprie de activare/dezactivare.

Atentie: Orice instructiune care reseteaza bitul EA („CLR EA”) trebuie urmată **imediat** de o instructiune al carei format are cel putin doi octeti. Daca are loc o intrerupere in timp ce este executata resetarea bitului EA si instructiunea urmatoare dureaza o singura perioada de ceas, este posibil ca aceasta sa fie considerata valida si rutina de deservire sa fie executata. Cu toate acestea, citirea bitului EA va returna valoarea 0 in interiorul rutinei. Cand instructiunea de resetare a lui EA este urmată de o instructiune ce dureaza mai multi cicli, intreruperea nu este considerata valida.

Unele dintre fanioanele de intrerupere sunt resetate hardware imediat ce este inceputa executia rutinei de deservire a intreruperii. Insa majoritatea acestora trebuie resetate software, inainte de iesirea din ISR. Daca un fanion ramane setat dupa ce a fost executata instructiunea de iesire din intrerupere (RETI), o noua cerere de intrerupere va fi generata imediat si nucleul de procesare va reintra in ISR dupa terminarea instructiunii curente.

2.1.1 Procesarea intreruperilor

Atunci cand are loc o intrerupere, microcontrolerul executa urmatoarea secventa de pasi inainte de a deservi cererea de intrerupere primita:

- Instructiunea curenta isi termina executia.
- Numaratorul de program este salvat in stiva, iar pagina SFR este salvata in stiva hardware pentru paginile SFR.
- Starea curenta a intreruperilor este salvata intern.
- Intreruperile cu aceeasi prioritate sunt blocate.

- Se incarca in numaratorul de program adresa vectorului de intrerupere.
- Se executa rutina de deservire a intreruperii.
- Rutina de deservire a intreruperii se termina cu instructiunea RETI.
- Se extrage din stiva hardware registrul SFR_PAGE, se incarca in PC adresa de program salvata in stiva software a microcontrolerului.
- Se activeaza intreruperile cu aceeasi prioritate.

2.1.2 Surse si vectori de intrerupere

Microcontrolerul are 20 de surse de intreruperi. O intrerupere poate fi simulata software prin setarea fanionului de intrerupere aferent. Daca intreruperea este activata, va fi generat un IRQ, iar procesorul va executa ISR-ul de la adresa asociata acelei intreruperi. Sursele de intrerupere, adresele vectorilor asociati, ordinea prioritatilor si bitii de control sunt prezentati in Tabelul 12.4 (fisierul C8051F04xRev1_4.pdf, pag. 155).

2.1.3 Setarea prioritatilor

Fiecare sursa de intrerupere poate fi avea doua nivele de prioritate: redusa si marita. O intrerupere cu prioritate redusa poate fi dezactivata temporar de o intrerupere cu prioritate marita. O intrerupere cu prioritate marita nu poate fi dezactivata temporar de o alta intrerupere. Fiecare intrerupere are asociat un bit de setarea a prioritatii, aflat in registrele SFR (IE-EIP2). Prioritatea predefinita este cea redusa. Daca au loc simultan doua cereri de intrerupere, este deservita intrerupere cu prioritate marita. Daca cele doua intreruperi au aceeasi prioritate, o ordine prestabilita (a se consulta Tabelul 12.4 din fisierul C8051F04xRev1_4.pdf, pag. 155-156) este folosita pentru decizia de deservire.

2.1.4 Latenta intreruperilor

Timpul de raspuns la IRQ depinde de starea in care se afla procesorul in acel moment. Intreruperile aflate in asteptare sunt verificate si decodate in functie de prioritate la fiecare tact de ceas. Cel mai mic timp de raspuns are cinci cicli masina: unul pentru detectarea intreruperii si patru pentru executarea instructiunii de apel a ISR-ului (LCALL). Daca o intrerupere se afla in asteptare cand este executata instructiunea RETI, o singura instructiune este executata inainte de apelul (LCALL) rutinei de deservire. Astfel, cel mai mare timp de raspuns pentru o intrerupere are loc atunci cand procesorul executa instructiunea RETI urmata de instructiunea DIV. In acest caz, timpul de raspuns este de 18 cicli masina: unul pentru detectarea intreruperii, cinci pentru executarea instructiunii RETI, opt pentru executarea instructiunii DIV si patru pentru executarea instructiunii LCALL. Daca procesorul executa un ISR pentru o intrerupere cu nivel egal sau mai mare de prioritate, noua intrerupere nu va fi deservita pana nu se termina executarea ISR-ului curent, inclusiv instructiunea RETI si urmatoarea instructiune.

Registrele speciale folosite pentru configurarea intreruperilor sunt prezentate in tabelele de la paginile (fisierul C8051F04xRev1_4.pdf, pag. 158-163).

3 Problema

3.1 Formularea problemei

Sa se implementeze un sistem de comunicatie de voce intre doua microcontrolere C8051F040. Se cere sa se utilizeze interfata seriala si sistemul de intreruperi al acesteia.

3.2 Solutie posibila

3.2.1 Descrierea modulelor hardware

Vom prezenta in continuare o implementare a unui sistem avand la baza doua placi de dezvoltare cu microcontrolerul C8051F040 si patru module externe (doua de intrare, doua de iesire) identice cu cele utilizate in Lucrarea nr. 4. Pentru descrierea hardware a acestor module consultati Lucrarea nr. 4, capitolul 3.2.1.

Cele doua placi de dezvoltare sunt conectate prin intermediul unui cablu serial cross.

3.2.2 Descrierea algoritmului

Observatie: programul respecta structura generala prezentata in Lucrarea nr.1, capitolul 3.2.2.

In solutia propusa programul contine rutinele `main`, `init` si `UART1Irr`, care este rutina de raspuns la intreruperea generata de interfata seriala.

Rutina principala (`main`) apeleaza subrutina de initializare (`init`) apoi trece in zona etichetata `start`. In zona etichetata `start` se forteaza o intrerupere care amorseaza comunicatia. Urmeaza apoi instructiunea `jmp $` care reprezinta o bucla infinita ce nu executa nimic.

Bucla infinita este intrerupta atunci cand are loc un eveniment (receptie, sfarsit de transmisie). In acest caz, PC-ul (Program Counter-ul) este initializat cu valoarea vectorului de intrerupere corespunzator `UART1` (`0x00A3`). La aceasta adresa se afla o instructiune de salt catre rutina de raspuns la intreruperi (`UART1Irr`).

Rutina `UART1Irr` testeaza tipul intreruperii (receptie, sfarsit de transmisie) si, in consecinta, trece fie in zona etichetata `send`, fie in zona etichetata `receive`. La inceputul rutinei se salveaza contextul (`SFRPAGE`, `ACC`). La sfarsitul rutinei se reda contextul.

Zona etichetata `receive` preia ultimul octet de date primit si il transfera la `DAC0`. Astfel, esantionul de semnal este convertit din digital in analogic si redat in difuzor. De asemenea, in aceasta zona se reseteaza flagul de intrerupere `RI1` (`UART1` este pregatit pentru o noua receptie).

Zona etichetata `send` preia un esantion din semnalul analogic de la microfon prin intermediul `ADC2`. Esantionul convertit este citit din `ADC2` si scris in registrul buffer al `UART1`. Astfel, se incepe transmisia lui. In continuare se forteaza o noua conversie (citirea unor date noi de la microfon); aceste date vor fi transmise la urmatorul apel al subrutinei de raspuns la intrerupere. Observatie: flagul de intrerupere `TI1` se reseteaza inaintea inceperii transmisiei (sfarsitul acesteia va determina setarea lui si, deci, o noua intrerupere).

Subrutina de initializare

- dezactiveaza intreruperile
 - o pentru detalii vezi explicatiile din Lucrarea nr. 1.
- dezactiveaza watchdog timer-ul
 - o pentru detalii vezi explicatiile din Lucrarea nr. 1.
- activeaza crossbar-ul
 - o pentru detalii vezi explicatiile din Lucrarea nr. 1.
- configureaza portul `P1`
 - o pentru detalii vezi explicatiile din Lucrarea nr. 2.
- configureaza tensiunea de referinta
 - o pentru detalii vezi explicatiile din Lucrarea nr. 2.
- configureaza convertorul analog-digital
 - o pentru detalii vezi explicatiile din Lucrarea nr. 2.
- configureaza convertorul digital-analog
 - o pentru detalii vezi explicatiile din Lucrarea nr. 4.
- configureaza oscilatorul intern
 - o pentru detalii vezi explicatiile din Lucrarea nr. 4.
- configureaza `Timer1`

- o pentru detalii vezi explicatiile din Lucrarea nr. 3.
- configureaza UART1
- o pentru detalii vezi explicatiile din Lucrarea nr. 3.

4 Desfasurarea lucrarii

1. Se creaza un proiect nou cu aplicatia din anexa 5.1
2. Se completeaza spatiile punctate din program cu instructiunile necesare configurarilor mentionate in comentariile codului.
3. Se ruleaza programul astfel completat pas cu pas, urmarindu-se efectul asupra registrelor, punandu-se un breakpoint in dreptul vectorului de intrerupere.
4. Sa se modifice programul de la Lucrarea 4, Anexa 5.2, astfel incat sa se foloseasca intreruperea generata de Timer3 pentru inceperea unei conversii a ADC2 si transferul catre DAC0.
5. Sa se scrie un program care sa modifice Anexa 5.2 a Lucrarii 4, astfel incat:
 - ADC2 sa inceapa conversia la depasirea Timer 2,
 - transferul la DAC0 sa se faca in interiorul unui vector de intrerupere generat de terminarea conversiei pentru ADC2.
6. Sa se scrie un program care sa genereze un semnal dreptunghiular cu frecventa controlata prin intermediul tastelor A si D astfel incat sa se dubleze frecventa (o octava) la apasarea tastei D si sa se reduca la jumatate la apasarea tastei A , cu ajutorul intreruperilor timerelor.

5 Anexe

5.1 Codul aplicatiei voiceTransmission

```

;-----
; FILE NAME   : voiceTransmission.asm
; TARGET MCU  : C8051F040
; DESCRIPTION : interMicroProcessor voice transmission.
;
; NOTES:
;-----
; EQUATES
;-----

        $include (c8051f040.inc)                ; Include register definition file.

;-----
; RESET and INTERRUPT VECTORS
;-----

                cseg      AT 0x0000                ; Reset Vector
                ljmp      main                    ; Locate a jump to the start of code
                                                ; at the reset vector.
                cseg      AT 0x00A3                ; UART1 interrupt vector
                ljmp      UART1Irr                ; Jump at the interrupt response routine

;-----
; MAIN PROGRAM CODE SEGMENT
;-----

mainCodeSeg      segment      CODE

                rseg      mainCodeSeg            ; Switch to this code segment.
                using     0                      ; Specify register bank for the
                                                ; following program code.

main:            call      init

start:           setb      TI1                    ; Trigger the first interrupt
                jmp       $                      ; Wait for interrupts

;-----
; FUNCTION CODE
;-----

init:
                ...                ; Disable global interrupts
                ...                ; Disable Watch Dog Timer
                ...

initIOAndCross: ...                ; Use SFRs in the
                ...                ; configuration Page
                ...                ; Enable the crossbar and UART 1
                ...                ; Configure P1.0 as analog input

initVREF:        ...                ; Use SFRs on the 0x00 Page
                ...                ; ADC2 voltage reference from internal VREF
                ...                ; Enable Bias Generator

initADC2:        ...                ; Use SFRs in the ADC2 Page
                ...                ; Set AIN1.0 as single-ended input
                ...                ; Select AIN1.0 as input channel for
                ...                ; the analog multiplexer
                ...                ; Configure a *1 Gain
                ...                ; Enable ADC2; Continuous tracking;

initDAC0:        ...                ; Use SFRs on the DAC0 Page
                ...                ; Enable DAC0
                ...                ; a conversion will start only after
                ...                ; a write in DAC0H; the data format used is 1xx.

initOscillator:  ...                ; set SYSCLK to 24.5 MHz

initTimer1:      ...                ; Use SFRs in Timer 1 Page
                ...                ; Use Timer 1 in 8-bit auto-reload mode
                ...                ; Set the reload value

```

```

        ...                ; Timer 1 will use the system clock
        ...                ; Enable Timer 1

initUART1:    ...                ; Use SFRs in UART 1 Page
        ...                ; Use UART 1 in 8-bit mode

initInterrupts:    mov        EIE2, #0x40        ; Enable and set high priority for
        mov        EIP2, #0x40        ; UART1 transmit/receive interrupt
        setb       EA                ; Enable all interrupts

        ret

;-----
; INTERRUPT SERVICE ROUTINES
;-----

UART1Irr:    push        SFRPAGE        ; Save the context (SFRPAGE and ACC)
        push        ACC
        jnb        RI1, send        ; Check the interrupt source:
                                ;end of transmission or receive

        receive:    mov        SFRPAGE, #UART1_PAGE
        mov        A, SBUF1        ; If data received, send it to DAC0
        clr        RI1        ; Clear the interrupt flag
        mov        SFRPAGE, #DAC0_PAGE
        mov        DAC0H, A        ; Write the data in DAC0H
        jmp        exit

        send:    mov        SFRPAGE, #ADC2_PAGE
        mov        A, ADC2        ; If data sent, read another byte
        clr        AD2INT        ;from the ADC2
        setb       AD2BUSY        ; Force a new conversion
        mov        SFRPAGE, #UART1_PAGE
        clr        TI1        ; Clear the interrupt flag
        mov        SBUF1, A        ; Sent the read data through the serial
                                ;connection

        exit:    pop        ACC
        pop        SFRPAGE        ; Restore the context (SFRPAGE and ACC)
        reti

;-----
; End of file.
END
    
```